

SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA1418

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool (BL3)

Assessment Tool Weightage: 20%

QUESTIONS:

Total Marks:50

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

I N-Samanth Reddy (192525083) (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines
specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

ERROR ANALYSIS TASK

Q1. Full Compiler Phase Error Analysis (BTL4) (10 Marks)

The following C program is intended to perform arithmetic operations, conditional checks, and looping constructs. However, it contains multiple errors across different phases of compilation including lexical, syntax, semantic, logical, and runtime errors.

```
#include <stdio.h>

intmain() {
    intnum1=25;
    floatx=3.5
    inty;

    y=num1+10

    if(y==35 {
        printf("Correct Value\n")
    }

    intarr[3] = {1,2,3};
    printf("%d\n", arr[3]);

    int*p;
    *p=20;

    intz=10/0;

    while(num1<30) {
        printf("%d\n", num1);
        num1++;
    }
    return0;
}
```

Tasks:

- Identify all errors with description
- Classify errors into lexical, syntax, semantic, logical, and runtime errors
- Explain which errors are detected during compilation and which occur at runtime
- Rewrite the corrected version of the program
- Analyze the impact of division by zero and pointer misuse
- Suggest improvements in compiler design for better error detection

Q2. Advanced Lexical Error Identification (BTL4) (10 Marks)

The following C program contains multiple lexical errors due to invalid identifiers, malformed constants, and incorrect symbols. Carefully analyze the program and answer the questions.

```
#include <stdio.h>

intmain() {
int5num=10;
floatvalue&=2.5;
charch='a';
inttotal=10#20;
int_data=15;
intdata@1=25;
printf("Sample Program);
inty=0xH12;
floatf=3.4.5;
charstr[] ="Hello;
return0;
}
```

Tasks:

- Identify all lexical errors with invalid tokens
- Explain which lexical rule is violated (identifier, constant, literal, symbol)
- Rewrite each invalid token into a valid token
- Explain how a lexical analyzer processes such errors
- Design a DFA rule to detect invalid identifiers starting with digits
- Suggest improvements in lexical error reporting

Q3. Intermediate Code and Optimization Analysis (BTL4) (10 Marks)

The following C program demonstrates arithmetic computation and optimization concepts but contains inefficiencies and errors.

```
#include <stdio.h>

intmain() {
    inta=5;
    intb=10;
    intc;

    c=a*b+a*b+b*a;

    if(a=5) {
        printf("Check\n");
    }

    intarr[2] = {1,2};
    printf("%d\n", arr[2]);

    int*ptr;
    *ptr=10;

    return0;
}
```

Tasks:

- Identify all errors and inefficiencies in the program
- Classify errors according to compiler phases
- Generate three-address code (TAC) for the expression
- Apply DAG representation to eliminate redundant computations
- Generate optimized target code with minimal register usage
- Analyze the runtime impact of array bounds violation and pointer misuse

① Full Compiler Phase Error Analysis with description

(a) Identify all errors

<u>Line</u>	<u>Error</u>	<u>Description</u>
float x=3.5	missing semicolon	• syntax error
y = num1 + 10	missing semicolon	• syntax error
if (y == 35) {	missing closing }	syntax error
printf ("correct value")	missing semicolon	Syntax error
arr[3]	Array index out of bounds	Runtime error
*p = 20;	Uninitialized pointer	Runtime error.
10/0	Division by zero	Runtime error.

② Classify the errors:

Lexical Errors

- None

Syntax Errors

- missing semicolon after float x=3.5
- missing semicolon after y = num1 + 10
- missing) in if (y == 35) {.
- missing semicolon after printf().

Semantic Errors

- None

Logical Errors

- None

Runtime Errors.

- Accessing arr[3]
- Dereferencing uninitialized pointer
- Division by zero.

(c) Compilation vs Runtime Detection

Compile - Time Errors.

- missing semicolons.
- missing parentheses.

Runtime Errors.

- Array index out of bounds.
- Pointer misuse
- Division by zero

(d) Correct Program

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int num1 = 25;
```

```
    float x = 3.5;
```

```
    int y;
```

```
    y = num1 + 10;
```

```
    if (y == 35)
```

```
    {
```

```
        printf("correct value\n");
```

```
    }
```

```
    int arr[3] = {1, 2, 3};
```

```
    printf("%d\n", arr[2]);
```

```
    int value = 20;
```

```
    int *p = &value;
```

```
    printf("%d\n", *p);
```

```
    while (num1 < 30)
```

```
    {
```

```
        printf("%d\n", num1);
```

```
        num1++;
```

```
    }
```

```
    return 0;
```

```
}
```


(e) Impact of Division by zero and pointer misuse

Division by zero:

- causes undefined behavior;
- Program may crash or terminate unexpectedly.

Pointer misuse:

- Dereferencing on uninitialized pointer may cause segmentation fault.
- can corrupt memory.

(f) Improvements in compiler design

- Better syntax error recovery.
- Detailed error messages with line numbers.
- static analysis for array bounds.
- warnings for uninitialized pointers.
- compile-time detection of constant division by zero.

② Advanced Lexical Error Identification

a) Identify lexical errors:

Errors

In void Toten

5num

value\$

10#20

data@1

Sample Program

0x#12

3.4.5

"Hello

Identifier starts with digit

invalid character '\$'

invalid symbol '#'

invalid '@'

missing closing quote

-n.

invalid hexadecimal constant

invalid floating constant

missing closing quote

⑥ Violated Lexical Rules:

Token

5num

value \$ \$ \$

10#20

data@1

"Sample Program

0x#12

3.4.5

"Hello

Rule Violated

identifier rule

invalid identifier

invalid operator

invalid identifier

String literal

Hexadecimal constant

Floating constant

String literal

(c) correct Tokens

incorrect

5num

Value

10#20

"Sample Program

0xH12

3.4.5.

"Hello

correct

nums

Value.

10+20

"Sample Program"

0x12

3.45

"Hello".

(d) Lexical Analyzer processing

- Reads input character by character.
- forms Tokens.
- matches Tokens with lexical rules.
- Reports invalid tokens.
- continues scanning remaining program.

(e) DFA Rule:

Start → Digit (0-9) — Reject state — Start — Letter —

→ Accept Identifier — Accept Identifier — letter

Digit — → Accept Identifier.

(f) improvements on Lexical Error Reporting:

- Display exact line number.
- Highlight invalid token.
- Suggest nearest valid token.

⑤ Intermediate Code and Optimization Analysis

① Errors and inefficiencies.

Errors.

- If $(a=5)$ used assignment instead.
- $arr[2]$ access outside array.
- pointer ptr is uninitialized.

Inefficiencies.

- Expression $a*b$ computed three times.

② Compiler phase classification

Error

if $(a=5)$

$arr[2]$

Pointer misuse

Repeated multiplication

Phase

Semantic/Logical

Runtime

Runtime

Optimization.

③ Three Address Code (TAC)

original Expression.

$$c = a*b + a*b + b*a.$$

Three Address Code:

$$t_1 = a*b$$

$$t_2 = a*b$$

$$t_3 = b*a$$

$$t_4 = t_1 + t_2$$

$$t_5 = t_4 + t_3$$

$$c = t_5$$

② DAG Optimization:

Since,

$$a * b = b * a$$

compute only once.

optimized DAG

$$t_1 = a * b$$

$$t_2 = t_1 + t_1$$

$$c = t_2 + t_1$$

$$c = 3 * (a * b)$$

③ Optimized Target code:

mov R₁, a

mov R₂, b

MUL R₁, R₂

MOV R₃, R₁

ADD R₃, R₁

ADD R₃, R₁

mov C, R₃

④ Runtime Impact:

Array Bounds Violation

- Accesses invalid memory.
- Produces garbage values.
- may crash program.

Pointer misuse:

- causes segmentation fault.
- may corrupt memory.
- Result in undefined behavior.